

Experiment-01 Home Tasks

Tasks:

- A. [Experiment - 1C] A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`. Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements:

- Every pizza must contain exactly 3 different toppings.
- The same topping cannot appear more than once in a pizza.
- Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

- Display the topping names separated by commas.
- Display the total ingredient cost of each pizza.
- Sort the output:
- First by highest total cost
- If two pizzas have the same cost, sort them alphabetically.

- B. [Experiment – 1D] Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements:

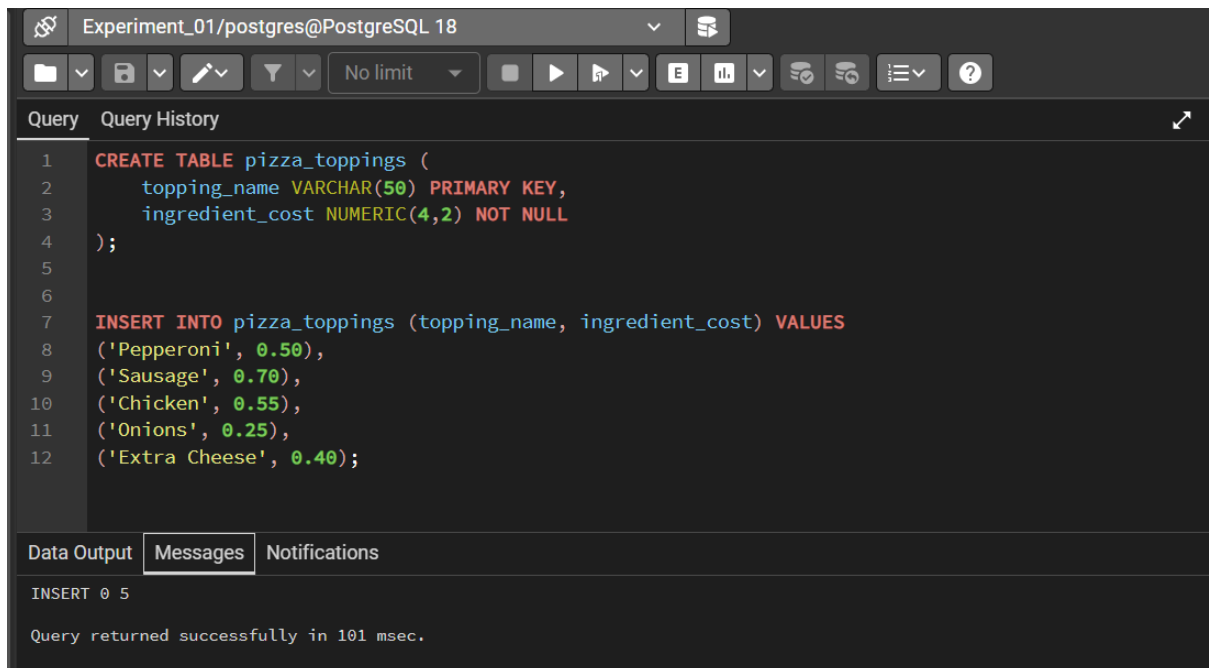
- Compare purchases made by the same user only.
- Ignore comparisons where the same transaction is matched with itself.
- The second purchase must occur after the first purchase.
- Same-day purchases must not be considered.
- The second purchase must occur within 7 days of the first purchase.
- Display only the unique User IDs that satisfy the above conditions.
- Sort the output in ascending order of `user_id`.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Code and Output:

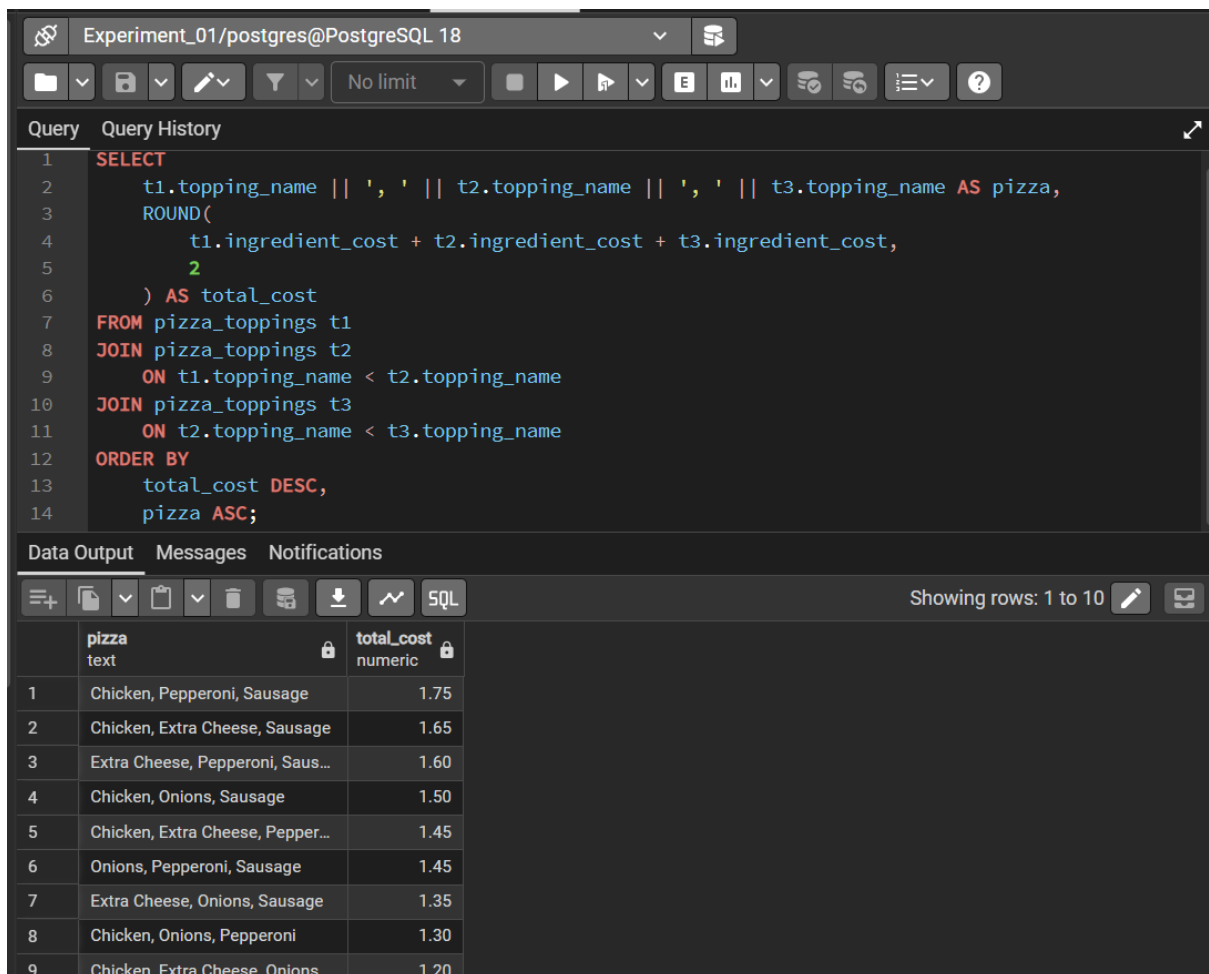
A.



The screenshot shows a PostgreSQL query editor window titled "Experiment_01/postgres@PostgreSQL 18". The query is as follows:

```
1 CREATE TABLE pizza_toppings (  
2     topping_name VARCHAR(50) PRIMARY KEY,  
3     ingredient_cost NUMERIC(4,2) NOT NULL  
4 );  
5  
6  
7 INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES  
8 ('Pepperoni', 0.50),  
9 ('Sausage', 0.70),  
10 ('Chicken', 0.55),  
11 ('Onions', 0.25),  
12 ('Extra Cheese', 0.40);
```

The interface includes tabs for "Query", "Query History", "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing the message "INSERT 0 5" and "Query returned successfully in 101 msec."



The screenshot shows the same PostgreSQL query editor window with a new query:

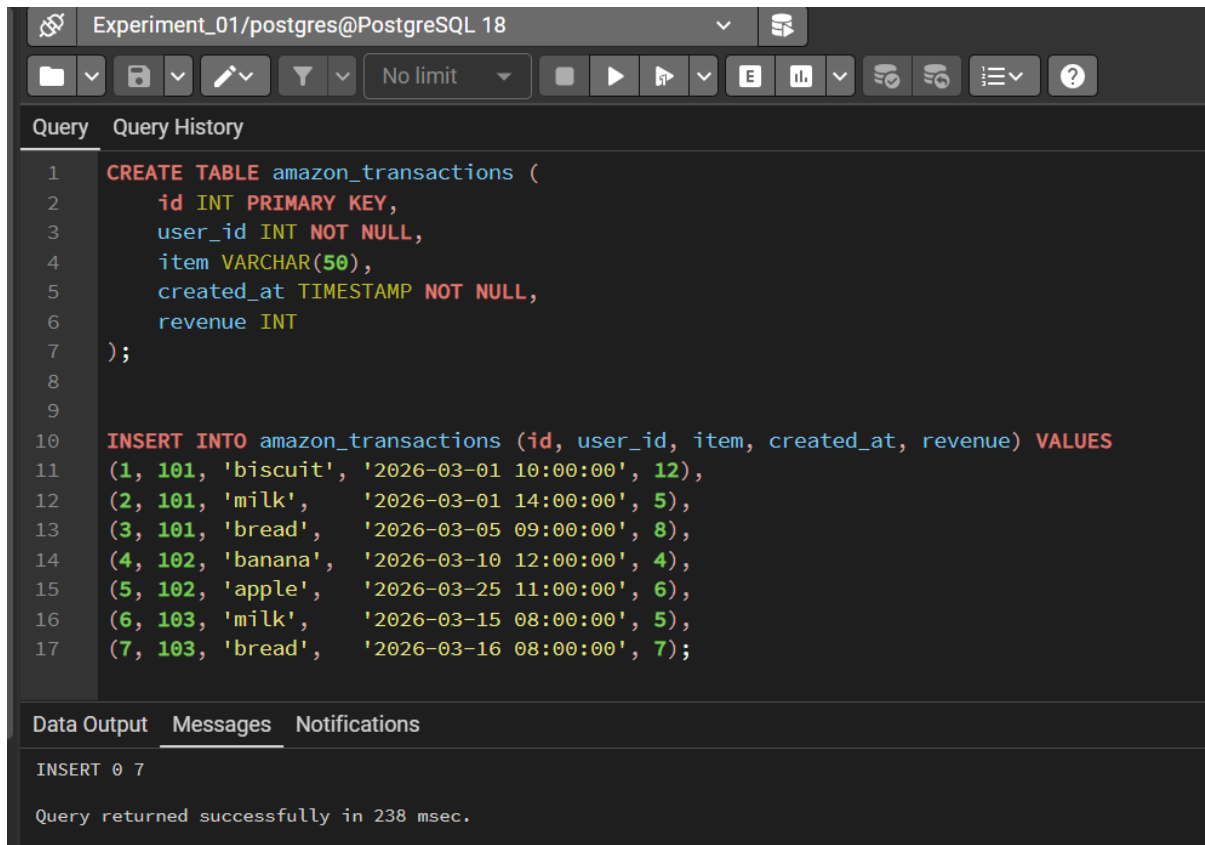
```
1 SELECT  
2     t1.topping_name || ', ' || t2.topping_name || ', ' || t3.topping_name AS pizza,  
3     ROUND(  
4         t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost,  
5         2  
6     ) AS total_cost  
7 FROM pizza_toppings t1  
8 JOIN pizza_toppings t2  
9     ON t1.topping_name < t2.topping_name  
10 JOIN pizza_toppings t3  
11     ON t2.topping_name < t3.topping_name  
12 ORDER BY  
13     total_cost DESC,  
14     pizza ASC;
```

The "Data Output" tab is active, displaying the results of the query. The output is a table with two columns: "pizza" (text) and "total_cost" (numeric). The results are sorted by total cost in descending order.

	pizza	total_cost
1	Chicken, Pepperoni, Sausage	1.75
2	Chicken, Extra Cheese, Sausage	1.65
3	Extra Cheese, Pepperoni, Saus...	1.60
4	Chicken, Onions, Sausage	1.50
5	Chicken, Extra Cheese, Pepper...	1.45
6	Onions, Pepperoni, Sausage	1.45
7	Extra Cheese, Onions, Sausage	1.35
8	Chicken, Onions, Pepperoni	1.30
9	Chicken, Extra Cheese, Onions	1.20

The interface also shows a toolbar with various icons and a status bar indicating "Showing rows: 1 to 10".

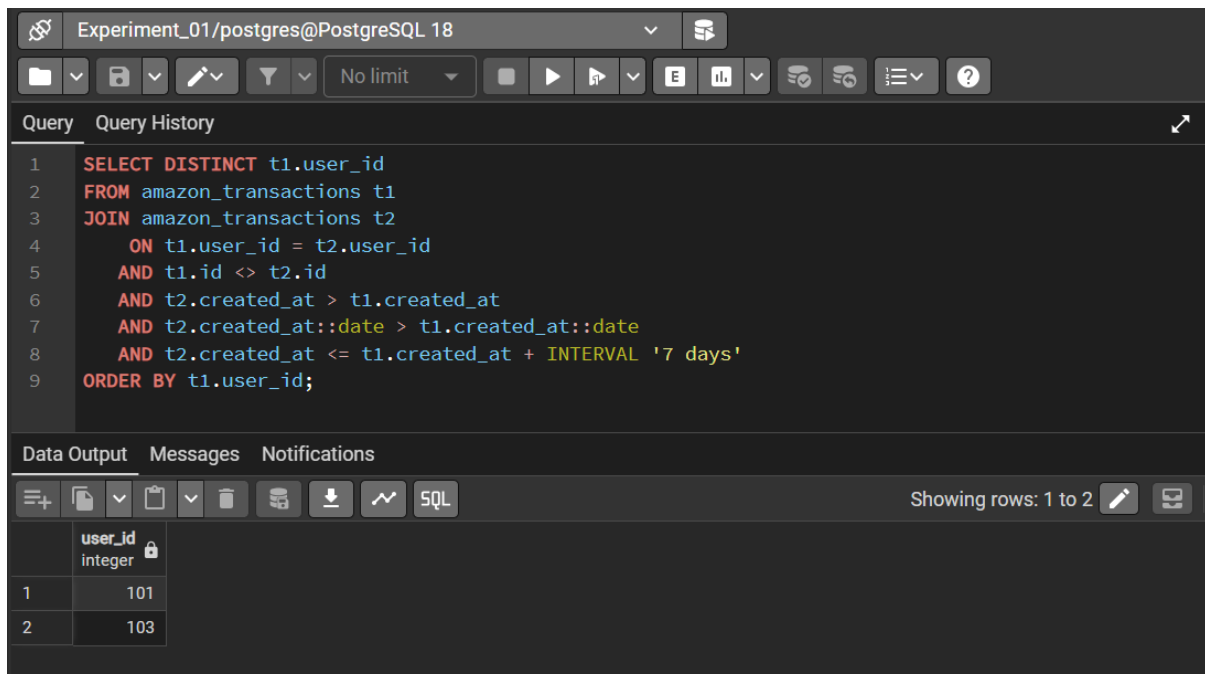
B.



The screenshot shows the PostgreSQL IDE interface. The top bar indicates the connection is 'Experiment_01/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying the following SQL script:

```
1 CREATE TABLE amazon_transactions (  
2     id INT PRIMARY KEY,  
3     user_id INT NOT NULL,  
4     item VARCHAR(50),  
5     created_at TIMESTAMP NOT NULL,  
6     revenue INT  
7 );  
8  
9  
10 INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES  
11 (1, 101, 'biscuit', '2026-03-01 10:00:00', 12),  
12 (2, 101, 'milk', '2026-03-01 14:00:00', 5),  
13 (3, 101, 'bread', '2026-03-05 09:00:00', 8),  
14 (4, 102, 'banana', '2026-03-10 12:00:00', 4),  
15 (5, 102, 'apple', '2026-03-25 11:00:00', 6),  
16 (6, 103, 'milk', '2026-03-15 08:00:00', 5),  
17 (7, 103, 'bread', '2026-03-16 08:00:00', 7);
```

The 'Data Output' tab is selected, showing the message: 'INSERT 0 7' and 'Query returned successfully in 238 msec.'



The screenshot shows the PostgreSQL IDE interface. The top bar indicates the connection is 'Experiment_01/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying the following SQL script:

```
1 SELECT DISTINCT t1.user_id  
2 FROM amazon_transactions t1  
3 JOIN amazon_transactions t2  
4     ON t1.user_id = t2.user_id  
5     AND t1.id <> t2.id  
6     AND t2.created_at > t1.created_at  
7     AND t2.created_at::date > t1.created_at::date  
8     AND t2.created_at <= t1.created_at + INTERVAL '7 days'  
9 ORDER BY t1.user_id;
```

The 'Data Output' tab is selected, showing the results of the query. The first column is 'user_id' (integer). The results are:

	user_id
1	101
2	103

The status bar at the bottom indicates 'Showing rows: 1 to 2'.

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with JOINS in PostgreSQL.
- Understood the concept of JOINS.